Audited by CRED SHIELDS

CredShields

Go Codebase Audit

March 18, 2026 • Public Audit Report

Description

This document details the process and result of the Go Codebase audit performed by CredShields Technologies PTE. LTD. on behalf of Ryze Protocol between January 20th, 2026, and February 23rd, 2026. A retest was performed on March 13th, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Pratik Lagaskar (Lead Auditor), Shreyas Koli (Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Ryze Protocol

AICPA SOC | SCS Advocates OWASP OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

1. Executive Summary ——————————- **5**

1.1 Residual Risk Summary 6

1.2 Accepted Design Tradeoffs 6

1.3 State of Security 6

2. The Methodology ——————————- **8**

2.1 Preparation Phase 8

2.1.1 Scope 8

2.1.2 Documentation 9

2.1.3 Audit Goals 9

2.2 Retesting Phase 11

2.3 Vulnerability classification and severity 11

2.4 CredShields staff 13

3. Findings Summary ——————————- **14**

3.1 Findings Overview 14

3.1.1 Vulnerability Summary 14

5. Bug Reports —————————— **17**

5.1 Execution Relayer 17

Bug ID #01 [Fixed] 17

Redis Messages Never Acknowledged Leading to Duplicate Execution 17

Bug ID #02 [Fixed] 19

Missing Low-S Validation in Relayer Signature Forwarding 19

Bug ID #03 [Fixed] 20

Missing Rate Limiting Despite Configuration 20

Bug ID #04 [Fixed] 21

Gas Estimation Uses Zero Value But Transaction Includes Pyth Fee 21

Bug ID #05 [Fixed] 23

Missing CEX Price Freshness Validation in BatchProcessor 23

Bug ID #06 [Fixed] 24

Duplicate Intents Enqueued Despite DB Dedupe 24

Bug ID #07 [Fixed] 25

Double Close on Channels and Storage Leading to Panic 25

Bug ID #08 [Fixed] 27

Pyth Retry Loop Leaks Response Body; Sleep Ignores Context 27

Bug ID #09 [Fixed] 29

Unused requestedIDs Parameter in Pyth parsePriceUpdates 29

Bug ID #10 [Fixed] 31

RelayerPool Release on Closed Channel Can Panic 31
Bug ID #11 [Fixed] 32
Join Intents — Zero Recipient and minSharesOut=0 Allowed 32
Bug ID #12 [Fixed] 34
waitForReceipt Treats All Errors as Retryable 34
Bug ID #13 [Fixed] 35
GetIntentStatus Returns "pending" for Any ID When Storage Is Nil 35
Bug ID #14 [Fixed] 36
Nonce Verification TOCTOU and Increment-Before-Send 36
Bug ID #15 [Fixed] 38
SaveIntent Panics if AmountIn Is Nil (Latent) 38
Bug ID #16 [Fixed] 40
Health Check Does Not Check DB (Only Ethereum client + Redis) 40
5.2 External Liquidity Feeder 41
Bug ID #01 [Fixed] 41
Decimal Unit Mismatch in calculateAMMImpact 41
Bug ID #02 [Fixed] 43
MustFloat64() Panic Kills Service 43
Bug ID #03 [Fixed] 44
Out-of-Bounds Panic in collectReadyBandedData 44
Bug ID #04 [Fixed] 46
rollingWindowDuration Never Initialized 46
Bug ID #05 [Fixed] 47
stopChan Double-Close - Manager Cannot Restart 47
Bug ID #06 [Fixed] 48
Data Race on elm.bandConfig 48
Bug ID #07 [Fixed] 50
Missing defensive check for zero oracleMidPrice 50
5.3 Signed Price Server 52
Bug ID #01 [Fixed] 52
Unsubscribe semantics inverted (empty subscription == send all) 52
Bug ID #02 [Fixed] 54
WritePump tight loop when SendCh closed (no comma-ok) 54
Bug ID #03 [Fixed] 55
server.max_connections exists but is not enforced 55
Bug ID #04 [Fixed] 57
Config defaults override explicit false (cannot disable) 57
Bug ID #05 [Acknowledged] 58
BinanceUS endpoint overrides are ignored 58

Bug ID #06 [Fixed] 59
Precision loss and truncation in priceToWad (IEEE-754) 59
Bug ID #07 [Fixed] 60
consumeSignedPrices infinite loop when SignedPrices channel closed (no comma-ok) 60
Bug ID #08 [Fixed] 61
shutdown_timeout numeric/format mismatch (config silently ignored) 61
Bug ID #09 [Fixed] 62
token_mappings address not hex-validated 62
Bug ID #10 [Fixed] 63
WorkerPool.Stop() not idempotent 63
Bug ID #11 [Fixed] 64
Multiple configuration variables are defined but unused (dead config) 64
6. The Disclosure ——————————- 66

1. Executive Summary ----------

Ryze Protocol engaged CredShields to perform a Go codebase security audit from January 20th, 2026, to February 23rd, 2026. During this timeframe, 34 vulnerabilities were identified. **A retest was performed on March 13th, 2026, and all High/Critical issues have been remediated and retested. One low severity finding remains acknowledged.**

During the audit, 7 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Ryze Protocol" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Σ
Execution Relayer	1	3	4	5	3	**16**
External Liquidity Feeder Bot	1	1	3	1	1	**7**
Signed Price Server	0	1	0	8	2	**11**
Σ	**2**	**5**	**7**	**14**	**6**	**34**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in the Go Codebase scope during the testing window while abiding by the policies set forth by Ryze Protocol's team.

1.1 Residual Risk Summary

This section provides a consolidated view of all findings that were not fully remediated as of the retest on March 19, 2026. Readers are encouraged to review these items alongside the executive summary to form a complete picture of the protocol's current security posture.

The remaining findings are primarily of low severity or relate to design decisions acknowledged by the Ryze Protocol team. These items do not represent immediate critical vulnerabilities but should be considered in the context of the protocol's operational and configuration assumptions.

Bug ID	Severity	Title	Status
#05 (signed-price-server)	Low	BinanceUS Endpoint Overrides Are Ignored	Acknowledged

Note: All Critical, High, and Medium findings were fully remediated. The single open item is a Low severity configuration limitation acknowledged as a known operational constraint.

1.2 Accepted Design Tradeoffs

#05 (signed-price-server) — BinanceUS Endpoint Overrides Are Ignored: The team acknowledged this limitation without applying a patch. NewBinanceProvider currently overwrites config-provided Rest/Websocket endpoint values whenever the provider name does not match ProviderBinance, meaning BinanceUS operators cannot route through custom endpoints (proxies, region-specific URLs, failover targets). The risk is limited to reduced operational flexibility and has no direct security or funds impact. The team is aware and accepts this as a current operational constraint.

1.3 State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Ryze Protocol's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Ryze Protocol can implement both

manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Ryze Protocol can future-proof its security posture and protect its assets.

2. The Methodology ----------

Ryze Protocol engaged CredShields to perform an audit of their Go Codebase. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documentation, README files, architecture documents, configuration files (TOML, environment files), and inline comments across the Go codebase to gain a thorough understanding of each service's architecture, features, and functionalities. They examined all packages, exported functions, HTTP/WebSocket handlers, database interactions, Redis stream consumers, and blockchain integration points, and created a dependency map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive.

A testing window from January 20th, 2026, to February 23rd, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon and the **deployed contracts conform to the GitHub commits mentioned below**:

IN SCOPE ASSETS
Audited commit(s): • Execution-relayer: https://github.com/ryze-protocol/execution-relayer/tree/c5ffdac669c76d849be8ceb86716b739e1030cd6 • Signed-price-bot: https://github.com/ryze-protocol/signed-price-server/tree/719efe5adca5957fc848ee778eb956860e461c52 • External-liq-feeder: https://github.com/ryze-protocol/external-liq-feeder/tree/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c

Retested/fixed commit(s):

- Execution-relayer: https://github.com/ryze-protocol/execution-relayer/tree/39ced1dca359f4d44062c8db486911a19391ef2e
- Signed-price-bot: https://github.com/ryze-protocol/signed-price-server/tree/9d96299a8a9bef88ef7d4158bf72ac714e2d8db9
- External-liq-feeder: https://github.com/ryze-protocol/external-liq-feeder/tree/a9c4c49bdd2cf24dcdc8c223c22ef89f6f65c7aa

This audit reflects the security posture of the Ryze Protocol smart contracts at the specified commit hashes in scope. Any modifications made after the completion of the audit and retest are outside the scope of this assessment.

2.1.2 Documentation

Project README files, architecture documents, and configuration examples (TOML, environment files) were provided and reviewed. Inline code comments supplemented the understanding where formal documentation was not available.

2.1.3 Audit Goals

CredShields employs a combination of static analysis tools (e.g. go vet, staticcheck, gosec), automated scanners, and thorough manual review processes to deliver comprehensive Go codebase security audits. The majority of the audit involves manual inspection of the source code, guided by industry-standard security frameworks (OWASP, CWE/SANS Top 25) and an extended, self-developed checklist built from best practices for backend services, cryptographic implementations, and blockchain-interacting systems. The team focuses on deeply understanding each service's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities.

Based on the codebase architecture the following key audit focus areas were validated during this engagement:

- **Private key management and cryptographic correctness** — Secure handling and usage of ECDSA signing keys (secp256k1) across all three services; validation of EIP-191 message formatting, signature low-S normalization (EIP-2 compliance), and WAD-format price encoding. The audit confirmed issues including signature malleability from missing low-S validation and IEEE-754 precision loss in price-to-WAD conversion.
- **Blockchain transaction integrity** — Transaction building, gas estimation, nonce management, Pyth oracle fee handling, and on-chain execution correctness. The audit confirmed issues including gas estimation simulating with zero value despite Pyth fee requirements, nonce TOCTOU races between verification and execution, and decimal unit mismatches corrupting AMM impact calculations for mixed-decimal token pools.
- **Message queue reliability and idempotency** — Redis Streams acknowledgment lifecycle, duplicate intent prevention, and delivery guarantees. The audit confirmed issues including messages never being ACK'd after processing (leading to duplicate execution risk) and duplicate intents being enqueued despite database-level deduplication.
- **Concurrency safety and goroutine lifecycle** — Race conditions in worker pools, WebSocket hubs, channel close semantics, shutdown ordering, and shared state access. The audit confirmed multiple issues including double-close panics on channels and storage across all three services, data races on shared configuration structs, and tight busy-loops from missing comma-ok checks on closed channels.
- **Input validation and data integrity** — Validation of EIP-712 intents, CEX price freshness, oracle price sanity (zero/NaN/Inf guards), join intent parameters, and configuration values. The audit confirmed issues including missing CEX price staleness checks, unguarded MustFloat64() calls that panic on bad oracle prices, missing zero-price defensive checks, and token mapping addresses not validated as hex.
- **Denial of service resilience** — Rate limiting enforcement, WebSocket connection caps, resource exhaustion, and retry/backoff correctness. The audit confirmed issues including rate limiting being configured but never enforced, max_connections being defined but never applied, and HTTP response body leaks in retry loops.
- **Error handling and observability** — Proper error classification (retryable vs. permanent), health check completeness, status endpoint correctness, and graceful degradation. The audit confirmed issues including transaction receipt polling treating all errors as retryable,

status endpoints returning misleading results when storage is unconfigured, and health checks omitting database connectivity.

- **Configuration correctness** — Detection of dead config, misleading defaults, type mismatches, and config values that are parsed but never enforced. The audit confirmed issues including rolling window durations never being initialized (defeating time-based filtering), boolean defaults overriding explicit false values, duration format mismatches silently falling back to hardcoded values, and multiple configuration variables that are parsed but never used.
- **Arithmetic and numerical correctness** — Decimal normalization across mixed-precision token pairs, WAD-format calculations, EWMA smoothing, and scaling factor derivation. The audit confirmed issues including a systematic decimal unit mismatch in AMM impact calculations that drove scaling factors to their maximum for mixed-decimal pools, and hardcoded loop bounds contradicting dynamic band configuration.

2.2 Retesting Phase

Ryze Protocol is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational findings are opportunities for improvement and do not pose a direct risk to the system. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or cannot be exploited repeatedly, or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private data, degradation of service reliability, or bypass of intended access controls. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

High-severity vulnerabilities pose a significant risk to the system and the organization. They can result in unauthorized access, loss of funds for some users, leakage of private keys or sensitive credentials, denial of service, or execution of unintended on-chain transactions. They may or may not require specific conditions and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in loss of funds, compromise of signing infrastructure, unauthorized transaction execution, systematic corruption of on-chain data submissions, or full system compromise. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and CWE classification. Each asset section includes a summary highlighting the key risks and observations.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 34 security vulnerabilities were identified in the asset.

Execution Relayer (16 findings)

Vulnerability Title	Severity	Vulnerability Type	Status
Redis Messages Never Acknowledged Leading to Duplicate Execution	**Critical**	Data Integrity / Idempotency	Fixed
Missing Low-S Validation in Relayer Signature Forwarding	**High**	Signature Malleability	Fixed
Missing Rate Limiting Despite Configuration	**High**	DoS / Resource Exhaustion	Fixed
Gas Estimation Uses Zero Value But Transaction Includes Pyth Fee	**High**	Logic Error	Fixed
Missing CEX Price Freshness Validation in BatchProcessor	**Medium**	Data Validation (CWE-1215)	Fixed
Duplicate Intents Enqueued Despite DB Dedupe	**Medium**	Idempotency	Fixed
Double Close on Channels and Storage Leading to Panic	**Medium**	Resource Management	Fixed
Pyth Retry Loop Leaks Response Body; Sleep Ignores Context	**Medium**	Resource Leak / Cancellation	Fixed
Unused requestedIDs Parameter in Pyth parsePriceUpdates	**Low**	Dead Code / Missing Validation	Fixed
RelayerPool Release on Closed Channel Can Panic	**Low**	Concurrency / Resource Management	Fixed

Vulnerability Title	Severity	Vulnerability Type	Status
Join Intents — Zero Recipient and minSharesOut=0 Allowed	**Low**	Relayer Input Validation Gap	Fixed
waitForReceipt Treats All Errors as Retryable	**Low**	Logic / Error Handling	Fixed
GetIntentStatus Returns "pending" for Any ID When Storage Is Nil	**Low**	Logic / Misconfiguration	Fixed
Nonce Verification TOCTOU and Increment-Before-Send	**Informational**	Race Condition / Design Robustness	Fixed
SaveIntent Panics if AmountIn Is Nil (Latent)	**Informational**	Defensive Programming / Latent Defect	Fixed
Health Check Does Not Check DB (Only Ethereum Client + Redis)	**Informational**	Observability	Fixed

External Liquidity Feeder (7 findings)

Vulnerability Title	Severity	Vulnerability Type	Status
Decimal Unit Mismatch in calculateAMMImpact	**Critical**	Arithmetic / Normalization	Fixed
MustFloat64() Panic Kills Service	**High**	Unhandled Panic / Service Crash	Fixed
Out-of-Bounds Panic in collectReadyBandedData	**Medium**	Deterministic DoS / Panic	Fixed
rollingWindowDuration Never Initialized	**Medium**	Logic Error	Fixed
stopChan Double-Close — Manager Cannot Restart	**Medium**	Lifecycle Bug / Latent Panic	Fixed
Data Race on elm.bandConfig	**Low**	Concurrency / Data Race	Fixed
Missing Defensive Check for Zero oracleMidPrice	**Informational**	Defensive Programming / Robustness	Fixed

Signed Price Server (11 findings)

Vulnerability Title	Severity	Vulnerability Type	Status
Unsubscribe Semantics Inverted (empty subscription == send all)	**High**	Logic Error	Fixed
WritePump Tight Loop When SendCh Closed (no comma-ok)	**Low**	Concurrency Bug	Fixed
server.max_connections Exists But Is Not Enforced	**Low**	Config / Operational	Fixed
Config Defaults Override Explicit False (cannot disable)	**Low**	Config Error	Fixed

Vulnerability Title	Severity	Vulnerability Type	Status
BinanceUS Endpoint Overrides Are Ignored	**Low**	Config Error	Acknowledged
Precision Loss and Truncation in priceToWad (IEEE-754)	**Low**	Incorrect Calculation / Decimal Normalization	Fixed
consumeSignedPrices Infinite Loop When Channel Closed (no comma-ok)	**Low**	Concurrency Bug	Fixed
shutdown_timeout Numeric/Format Mismatch (config silently ignored)	**Low**	Config Semantics	Fixed
token_mappings Address Not Hex-Validated	**Low**	Missing Validation / Config Footgun	Fixed
WorkerPool.Stop() Not Idempotent	**Informational**	Logic Error	Fixed
Multiple Configuration Variables Defined But Unused (Dead Config)	**Informational**	Dead Code	Fixed

Table: Findings and Remediations

5. Bug Reports ----------

5.1 Execution Relayer

Bug ID #01 [Fixed]

Redis Messages Never Acknowledged Leading to Duplicate Execution

Vulnerability Type
Data Integrity / Idempotency

Severity
Critical

Description
Messages are dequeued from Redis Streams but never ACK'd after successful processing. The `batchCollector` function extracts intents from `ConsumeResult` objects but discards the `MessageID` field, and `handleResult()` never calls `queue.Ack()` for successful executions or `queue.Nack()` for failures.

This means messages remain in Redis Streams Pending Entries List (PEL) indefinitely. On consumer restart, messages won't be redelivered because Dequeue uses ">" (new messages only), but the pending messages will never be cleaned up. After idle timeout, these messages could be claimed by other consumers and re-executed.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/executor.go#L252-L295
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/executor.go#L399-L468

Impacts
Messages remain in Redis' pending state indefinitely because they are never acknowledged after processing, creating a critical data integrity issue.
When the consumer restarts or experiences idle timeout conditions, these pending messages may be reclaimed and reprocessed, causing duplicate intent execution where the same swap might execute multiple times.

Remediation

Pass `MessageID` through the processing pipeline and call `queue.Ack()` on success and `queue.Nack()` (or equivalent) on failure.

Retest

Fixed. 0fc320d, PR #7. `MessageIDs` preserved in batch; handleResult calls `queue.Ack()` on success and `queue.Nack()` on failure. PR #23 (1d92c71) later: reverted TXs are Acked (not Nack) so not retried; `DecodeRevertReason` added.

Bug ID #02 [Fixed]

Missing Low-S Validation in Relayer Signature Forwarding

Vulnerability Type
Signature Malleability

Severity
High

Description
The relayer does not validate that signature S values are in the lower half of the secp256k1 curve order (s <= secp256k1n/2) before forwarding signatures to the contract. This creates a mismatch between off-chain and on-chain signature validation. The relayer uses `crypto.SigToPub()` which accepts both low-S and high-S signatures, but the contract uses OpenZeppelin's `ECDSA.recover()` (except: `SSPCore.sol` which uses raw `ecrecover`); which enforces low-S (EIP-2). High-S signatures pass relayer verification but fail on-chain, causing transaction reverts and wasted relayer gas.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/pkg/eip712/intent_hash.go#L145C4-L173
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L173-L178

Impacts
Users see "verification passed" in relayer logs but execution fails on-chain, creating poor user experience and confusion. Attackers could submit high-S signatures to waste relayer funds on failed transactions, creating a potential DoS vector.

Remediation
Add low-S validation in `signatureToBytes` or create a validation function called before encoding.

Retest
Fixed. 55187bd, PR #8. Low-S validation added before forwarding; high-S signatures rejected so relayer and contract agree.

Bug ID #03 [Fixed]

Missing Rate Limiting Despite Configuration

Vulnerability Type
DoS / Resource Exhaustion

Severity
High

Description
The server configuration includes rate limiting options (`EnableRateLimit`, `RateLimitRequests`, `RateLimitDuration`) but no rate limiting middleware is actually implemented in the server setup. The configuration exists but is never checked or used, leaving the API completely unprotected against request flooding. Also, the default configuration sets `EnableRateLimit: true`, which is misleading since rate limiting is not enforced.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/server.go#L27-L29 (Config exists but not used)
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/server.go#L32-L67
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/server.go#L114-L115

Impacts
The API is completely vulnerable to DoS attacks where an attacker can flood the endpoint with unlimited intent submissions, overwhelming the relayer with more requests than it can reasonably process. The Redis queue can also be exhausted as unbounded intent submissions fill up the queue storage, potentially causing Redis to run out of memory or hit configured limits that prevent legitimate intents from being queued.

Remediation
Implement rate limiting middleware using Fiber's limiter.

Retest
Fixed. e81bac8, PR #4. Fiber limiter middleware added when `EnableRateLimit` is true; `RateLimitRequests` and `RateLimitDuration` from config applied.

Bug ID #04 [Fixed]

Gas Estimation Uses Zero Value But Transaction Includes Pyth Fee

Vulnerability Type
Logic Error

Severity
High

Description
Gas estimation is performed with `Value: big.NewInt(0)` before the Pyth fee is calculated, but the actual transaction includes `Value: pythFee`. When Pyth update data is present, the contract's `updatePriceFeeds()` function validates `msg.value >= updateFee` during gas estimation simulation.

Since the simulation uses `Value: 0` while the contract requires a fee, the validation fails and causes the gas estimation to revert. This causes gas estimation to fail entirely and fall back to the maximum gas limit (5M gas), preventing accurate gas estimation and the 20% buffer optimization from being applied.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L87-L90
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L97-L100
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L269-L290
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L109

Impacts
Gas estimation fails entirely when Pyth updates are present because the contract's fee validation reverts during simulation with `Value: 0`. The system falls back to the maximum gas limit (5M gas) instead of using an accurate estimate, preventing the 20% buffer optimization from being applied. Transactions succeed because the actual transaction includes the correct Pyth fee, but they always use excessive gas (5M instead of typical 1-2M), resulting in significant gas waste.

Thus, the relayer incurs higher transaction costs as it must maintain sufficient ETH balance to cover the maximum gas limit multiplied by the maximum fee per gas, rather than optimized estimates. This inefficiency affects every transaction that includes Pyth price updates, which is a common case for batches requiring oracle data, making the economic impact substantial over time.

Remediation

Reorder operations to calculate the Pyth fee before gas estimation and pass it to the estimation function.

Retest

Fixed. cf13c6e, PR #9. Pyth fee computed before gas estimation and passed into estimation call so simulation sees `Value >= updateFee`.

Bug ID #05 [Fixed]

Missing CEX Price Freshness Validation in BatchProcessor

Vulnerability Type
Data Validation (CWE-1215)

Severity
Medium

Description
The `ProcessBatch` function retrieves CEX prices from the price server but does not validate the freshness (staleness) of these prices before forwarding them for on-chain execution. A TODO comment explicitly acknowledges this gap. While the on-chain contract (SSPCore.sol) does have a `tTolerance` check, the off-chain relayer should reject stale prices earlier to prevent wasted gas and improve security posture.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/batch_processor.go#L80-L90

Impacts
Without off-chain staleness validation, the relayer may submit transactions with expired CEX price data, causing predictable on-chain reverts. This results in wasted gas fees for the protocol and degraded user experience as intent execution is unnecessarily delayed until fresh prices are obtained.

Remediation
Add a configurable max age for CEX prices (eg. default 60s). In `ProcessBatch`, before using the prices, reject the batch if any price's timestamp is older than that limit and return a clear error so the batch can be retried with fresh prices.

Retest
Fixed. 4572ef8, PR #6. CEX price staleness check in batch processing; batch rejected if any price older than configured max age (e.g. `tau`).

Bug ID #06 [Fixed]

Duplicate Intents Enqueued Despite DB Dedupe

Vulnerability Type
Idempotency

Severity
Medium

Description
`SaveIntent` uses `ON CONFLICT (intent_id) DO NOTHING` but does not return whether a row was actually inserted (no `RETURNING`, no `RowsAffected`). `VerifyAndEnqueue` always calls `Enqueue` after `SaveIntent` regardless of insert result. A duplicate submission (same intent_id, e.g. client retry or replay) therefore still enqueues. The executor will process it again; the second execution will hit the contract's nonce and revert, wasting relayer gas and polluting status.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L104-L113
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/storage.go#L179-L223

Impacts
Duplicate submissions with the same `intent_id` are still enqueued and executed. The second execution reverts on-chain (nonce already used), wasting relayer gas and polluting status/DB. There is no idempotency guarantee for duplicate POSTs.

Remediation
Have `SaveIntent` return whether the row was actually inserted (e.g. `RETURNING intent_id` or check `RowsAffected`) and only call `Enqueue` when the intent is new. Alternatively, call `GetIntent` before enqueue and skip enqueue if the intent already exists.

Retest
Fixed. 9b6344e, PR #10. `SaveIntent` returns (inserted bool, err); `VerifyAndEnqueue` only calls `Enqueue` when inserted is true; duplicates not enqueued.

Bug ID #07 [Fixed]

Double Close on Channels and Storage Leading to Panic

Vulnerability Type
Resource Management

Severity
Medium

Description
`Stop()` and `Close()` on the executor, relayer pool, and CEX price client close channels without guard (no `sync.Once` or "already closed" check). A second call panics ("close of closed channel"). In addition, storage is closed in two places: in `main` (defer `storage.Close()`) and inside `BatchExecutor.Stop()` (`e.storage.Close()`). On graceful shutdown, `Stop()` runs and closes storage; then when `main` exits, `defer storage.Close()` runs again. Closing an already-closed `*sql.DB` can cause panics or undefined behavior depending on the driver.

Affected Code

- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/executor.go#L203-L212
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/relayer_pool.go#L379-L382
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/cmd/api-server/main.go#L46
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/cmd/api-server/main.go#L81
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/cmd/api-server/main.go#L153

Impacts
A second call to `Stop()` or `RelayerPool.Close()` can panic and crash the process. On every normal shutdown, storage is closed twice (in `Stop()` and again in `main`'s defer), which can panic or cause undefined behavior depending on the DB driver. Shutdown is unreliable and may lead to connection pool or data integrity issues.

Remediation

Use `sync.Once` (or equivalent) for each channel and storage close. Ensure storage is closed in exactly one place (e.g. only in `main` and remove `e.storage.Close()` from `Stop()`, or the reverse, but not both).

Retest

Fixed. d4bc180, PR #11. `sync.Once` for executor/relayer pool channel close; storage closed only in main (removed from `Stop()`); no double close.

Bug ID #08 [Fixed]

Pyth Retry Loop Leaks Response Body; Sleep Ignores Context

Vulnerability Type
Resource Leak / Cancellation

Severity
Medium

Description
Two issues in `doRequestWithRetry`:
(1) Response body leak: `defer resp.Body.Close()` is inside the retry loop. In Go, defers run when the function returns, not when a loop iteration continues. On `continue` after a successful `Do(req)` (e.g. after body read error at L106 or retryable status at L116), the current response body is never closed before the next iteration. Each such retry leaks one response body; under repeated failures this can exhaust file descriptors or memory.
(2) Backoff ignores context: Backoff uses `time.Sleep(backoff)` without listening to `ctx.Done()`, so shutdown or request cancellation is delayed until the sleep completes.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/oracle/pyth/client.go#L74-L134
- http://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/oracle/pyth/client.go#L104-L120

Impacts
Each retry that continues without returning leaks one response body; under load this can exhaust file descriptors or memory. Backoff uses uninterruptible sleep, so shutdown and request cancellation are delayed. Resilience under failure and during graceful shutdown is poor.

Remediation
Close the response body explicitly before each `continue` in the error branches (do not rely on defer inside the loop). Use context-aware backoff (e.g. `select { case <-time.After(backoff): case <-ctx.Done(): return ... }` or a context-aware sleep helper).

Retest

Fixed. 37c447a, PR #14. Response body closed explicitly before `continue` in retry loop; context-aware sleep (select on `ctx.Done()`) during backoff.

Bug ID #09 [Fixed]

Unused `requestedIDs` Parameter in Pyth `parsePriceUpdates`

Vulnerability Type
Dead Code / Missing Validation

Severity
Low

Description
The `parsePriceUpdates()` function accepts a `requestedIDs []string` parameter but never uses it to validate or filter the returned price feeds. The function creates `PriceUpdate` objects for all entries in `resp.Parsed` without checking that those feed IDs match the original `priceFeedIDs` passed into `GetPriceUpdates()`. This means the function cannot detect if Hermes returns unexpected feeds, missing feeds, or extra feeds that weren't requested.

While the single Merkle accumulator root in the VAA cryptographically binds exactly the feeds in that update (making this safe from a protocol perspective), the unused parameter suggests an incomplete implementation where validation was intended but never completed. This leaves a gap in defensive programming where the relayer trusts Hermes completely without any validation layer.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/oracle/pyth/client.go#L42-L73
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/oracle/pyth/client.go#L138-L164

Impacts
This creates a trust boundary issue where the relayer must completely trust Hermes without any validation layer. If Hermes has a bug or is compromised, the relayer would accept and use incorrect price feeds as is. So, the relayer has no way to detect if Hermes returns unexpected feeds, missing feeds, or extra feeds that weren't requested.

Remediation
If the design intentionally trusts Hermes and the VAA Merkle root binding, remove the unused parameter to make the design intent explicit.

Retest

Fixed. fe062d3, PR #12. Unused `requestedIDs` parameter removed from `parsePriceUpdates` (dead code cleanup); design explicitly trusts Hermes/VAA binding.

Bug ID #10 [Fixed]

RelayerPool Release on Closed Channel Can Panic

Vulnerability Type
Concurrency / Resource Management

Severity
Low

Description
`Release(relayer)` sends on `rp.available`. In Go, sending on a closed channel panics. After `Close()`, that channel is closed; any later `Release` (e.g. slow worker, shutdown race, or test) would send on a closed channel and panic. Under current shutdown order (`Stop()` calls `wg.Wait()` before `RelayerPool.Close()`, so workers release before the pool is closed) the panic typically does not occur in production. The finding is that the pool API does not guard against `Release` after `Close()`; any change to shutdown order or any test/future caller that releases after close can trigger a panic.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/relayer_pool.go#L206-L215
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/relayer_pool.go#L379-L382

Impacts
If `Release` is ever called after `Close()` (e.g. slow worker or shutdown race), the process panics. The `Release` contract is unsafe: tests or future code can easily trigger this. There is no defense if shutdown ordering changes.

Remediation
Use `sync.Once` in `Close()` and in `Release` check a "closed" flag (or equivalent); if the pool is closed, skip sending on the channel (e.g. no-op or log and return).

Retest
Fixed. 49f7bd6, PR #13. `RelayerPool.Release` checks closed state (select on rp.done); if pool closed, no send on available channel; avoids panic.

Bug ID #11 [Fixed]

Join Intents — Zero Recipient and minSharesOut=0 Allowed

Vulnerability Type
Relayer Input Validation Gap

Severity
Low

Description
For join intents, validation does not require (1) non-zero `Recipient`, or (2) strictly positive `MinSharesOut`. Swap intents require non-zero recipient and positive `MinAmountOut`; the join branch does not. Zero recipient is allowed; `MinSharesOut` may be nil or zero (only negative is rejected via `MinSharesOut.Sign() < 0`). Such intents are accepted and enqueued. The contracts revert on-chain (InvalidToken / InvalidAmount), so execution is safe; the relayer-side gap is gas waste and poor UX (clients see on-chain reverts instead of clear validation errors).

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L166-L188 (join branch: no Recipient or MinSharesOut check)
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L121 (swap branch: for comparison)
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/handler.go#L197-L256 (join path: recipient from request, minSharesOut defaults to 0 if omitted)

Impacts
Invalid join intents (zero recipient or minSharesOut=0) are accepted and enqueued; execution then reverts on-chain, wasting relayer gas. Clients get confusing on-chain reverts instead of clear validation errors. The contract enforces safety; the relayer validation gap is operational and UX-only.

Remediation
For join intents, require `Recipient != (common.Address{})` and validate `MinSharesOut` (e.g. positive or per contract). Align relayer validation with contract checks to avoid gas waste and clear errors.

Retest

Fixed. bf38a57, PR #15. Join intent validation: Recipient required; `MinSharesOut` required and positive (zero/nil rejected).

Bug ID #12 [Fixed]

waitForReceipt Treats All Errors as Retryable

Vulnerability Type
Logic / Error Handling

Severity
Low

Description
In `waitForReceipt` (tx_builder.go), the loop only exits when `receipt, err := client.TransactionReceipt(...)` returns `err == nil`. On any other error (e.g. invalid hash, RPC failure, context cancelled), the loop does not return; it keeps polling until the context times out. The code does not distinguish "tx not yet mined" (retryable) from permanent failures; both are treated as retryable, so the real error is lost and the caller waits until timeout.

Affected Code

- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L317-L337

Impacts
Permanent errors (invalid hash, RPC failure, context cancelled) are treated as "retryable," so the caller waits until timeout instead of failing fast. The real failure cause is lost and debugging is harder. Goroutines and connections stay held for the full timeout.

Remediation
In `waitForReceipt`, distinguish "not found" / "pending" from other errors and return the latter immediately with the underlying error instead of retrying until timeout.

Retest
Fixed. f21de70, PR #16. `waitForReceipt` distinguishes "not found/pending" (retry) from other errors (return immediately with underlying error).

Bug ID #13 [Fixed]

GetIntentStatus Returns "pending" for Any ID When Storage Is Nil

Vulnerability Type
Logic / Misconfiguration

Severity
Low

Description
When `v.storage == nil` (e.g. no DB configured), `GetIntentStatus` returns immediately with status `"pending"` for any intent ID. No lookup is performed; non-existent or invalid IDs are reported as pending. This is misleading and can hide misconfiguration or missing storage.

Affected Code

- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L259-L265

Impacts
When storage is nil, every `GetIntentStatus` returns 200 with status "pending" for any intent ID, including non-existent ones. Misconfiguration (missing DB) is hidden from operators and clients, who may believe intents are pending when they were never stored.

Remediation
When storage is nil, return an error or a distinct status (e.g. "unknown" or 503) instead of "pending", or document that status is unknown when storage is disabled.

Retest
Fixed. 94c6709, PR #17. `GetIntentStatus` returns error "intent status unavailable: storage not configured" when storage is nil instead of "pending".

Bug ID #14 [Fixed]

Nonce Verification TOCTOU and Increment-Before-Send

Vulnerability Type
Race Condition / Design robustness

Severity
Informational

Description
Current implementation is correct; the contract enforces user nonce on-chain (no wrong execution), and the relayer calls SyncNonce on all existing build/send failure paths, so relayer nonce stays in sync today. This finding is a defensive suggestion: it identifies a TOCTOU window and a fragile nonce-increment pattern that could cause operational impact or future regressions. Nonce is verified only at submit time, then the intent is enqueued. Between check and execution, another intent with the same user nonce can be submitted and also pass verification. Both get enqueued; only one will succeed on-chain and the rest revert. The contract prevents incorrect execution; the relayer does not re-check user nonce at execution time, so the TOCTOU window is unmitigated.

Affected Code
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/tx_builder.go#L81-L88
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/executor.go#L341-L347
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/executor.go#L357-L362

Impacts
In current implementation there is no incorrect execution; contract enforces nonce. Duplicate same-nonce intents are accepted and enqueued; only one succeeds on-chain and the rest revert, wasting relayer gas and confusing users (operational/UX).
Future risk: Any new failure path that skips SyncNonce could leave the relayer with a desynced nonce and stuck or incorrect transactions.

Remediation
Reduce TOCTOU by re-checking user nonce at execution time (e.g. before building the tx) or by reserving nonce at enqueue time. Keep SyncNonce on all build/send failure paths (and document

this requirement for future changes). Consider incrementing relayer nonce only after successful send to make the design robust against new failure paths.

Retest

Fixed. 6d436a8, PR #20. User nonces re-verified at execution time (`VerifyUserNonces` before `BuildAndSignTransaction`); `SyncNonce` kept on all failure paths.

Bug ID #15 [Fixed]

SaveIntent Panics if AmountIn Is Nil (Latent)

Vulnerability Type
Defensive Programming / Latent Defect

Severity
Informational

Description
`SaveIntent` calls `intent.AmountIn.String()` in the INSERT values without a nil check. If any caller ever passes an intent with nil `AmountIn`, the process panics. The only current caller is `VerifyAndEnqueue`, which runs after `validateBasicFields`; for both swap and join, `validateBasicFields` requires `intent.AmountIn != nil && intent.AmountIn.Sign() > 0`, so in the current API path the panic is not reachable. The defect is latent: any future caller (admin tool, test, or other API path) that passes nil `AmountIn` would trigger a panic.

Affected Code

- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/executor/storage.go#L215
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L137-L138
- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L184-L185

Impacts
Any caller that passes an intent with nil `AmountIn` into `SaveIntent` will trigger a panic. No current API path does this; the risk is from future callers, tests, or refactors. A defensive check would avoid accidental panics and make the API contract clear.

Remediation
At the start of `SaveIntent`, check `intent.AmountIn != nil` (and other required fields as appropriate) and return a clear error if nil; do not call `AmountIn.String()` without a guard.

Retest

Fixed. 6a43652, PR #18. `SaveIntent: nil` and required-field checks for `AmountIn` (and join/joinProportional) before .String(); returns clear errors.

Bug ID #16 [Fixed]

Health Check Does Not Check DB (Only Ethereum client + Redis)

Vulnerability Type
Observability

Severity
Informational

Description
`HealthCheck` calls `v.ethClient.BlockNumber(ctx)` and `v.queue.HealthCheck(ctx)`. There is no call to storage (e.g. `storage.Ping` or a simple query). When storage is configured, the health endpoint can still report healthy while the database is unavailable. Load balancers or orchestrators may keep sending traffic to an instance that cannot persist or read intent status.

Affected Code

- https://github.com/ryze-protocol/execution-relayer/blob/c5ffdac669c76d849be8ceb86716b739e1030cd6/internal/api/verifier.go#L297-L309

Impacts
The health endpoint can report healthy while the database is down. Operators may miss DB outages, and load balancers may keep sending traffic to an instance that cannot persist or update intent status.

Remediation
When storage is configured, include a storage health check (e.g. ping or lightweight query) in `HealthCheck` and fail health when the DB is unreachable.

Retest
Fixed. 1e1fd75, PR #19. `HealthCheck` calls `storage.HealthCheck` when storage is configured; health fails when DB is unreachable.

5.2 External Liquidity Feeder

Bug ID #01 [Fixed]

Decimal Unit Mismatch in `calculateAMMImpact`

Vulnerability Type
Arithmetic / Normalization / Wrong Scaling Factor Submission

Severity
Critical

Description
`calculateAMMImpact` simulates the AMM price impact of a trade to solve for the scaling factor q(B) so that AMM slippage matches CEX slippage (per the PRD). It reads pool balances from the on-chain `getPoolAsset` return and uses float64 arithmetic to derive execution price and price impact. But, there is a dimensional unit mismatch: the function computes `executionPrice` as the ratio of `amountOut` (quote token native ERC20 decimals) to `amountIn` (base token native ERC20 decimals), then divides by `oracleMidPrice` (human-readable, e.g. 3000.0 USDC/ETH).
When base (e.g. WETH, 18 decimals) and quote (e.g. USDC, 6 decimals) differ, the ratio `amountOut / amountIn` is not a human-readable price and carries a 10^(18−6) = 10^12 unit error.
Also, the contract side returns raw ERC20 balances (not WAD-normalized) from `getPoolAsset`. As a result, for mixed-decimal pools the AMM-side price impact is pegged at ~10,000 bps (100%) for every trade size; the binary search in `solveScalingFactor` converges to the smallest scaled notional and thus the largest possible scaling factor (MaxScalingFactor = 1000).
For pools where both tokens have the same decimals this unit bug does not fire, but for mixed-decimal pools it systematically drives scaling factors to their maximum and severely corrupts on-chain calibration.

Affected Code

- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L157-L197 (`calculateAMMImpact`: base/quote balances and weights; k in heterogeneous units; `executionPrice = amountOut/amountIn` (or inverse) in raw units; `priceImpact = Abs(((executionPrice / oracleMidPrice) - 1) * 10000)` without decimal normalization.)
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L201-L229

(`solveScalingFactor` uses the wrong impact and propagates to scaling factor and on-chain submission.)

- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity_test.go (Tests use 18 decimals for both tokens, masking the bug for production WETH/USDC (18/6))

Impacts

Scaling factors are systematically wrong for pools with differing token decimals (e.g. ETH/USDC). All submitted values sit at or near `MaxScalingFactor = 1000`, so the protocol applies incorrect external liquidity ratios and misprices trades. The bug triggers under normal operation with no attacker path; economic impact falls on traders.

Remediation

Normalize `amountOut` and `amountIn` to the same decimal basis before computing `executionPrice` (e.g. both to 18 decimals/WAD, or to human units using 10^decimals).

Retest

Fixed. 305c261, PR #7. Base/quote amounts normalized to same decimal basis before `executionPrice` and price impact in `calculateAMMImpact`/`solveScalingFactor`.

Bug ID #02 [Fixed]

`MustFloat64()` Panic Kills Service

Vulnerability Type
Unhandled Panic / Service Crash /Latent error

Severity
High

Description
The entry point for all external liquidity computation calls `tPrice.Price.MustFloat64()` to convert the oracle price to float64. The `Must*` convention in Go means the function panics on error instead of returning an error.
If the price is NaN, infinite, out of float64 range, or otherwise unrepresentable, this panics. The panic is not recovered anywhere in the call stack (no `recover()` in `ProcessExternalLiquidity` or its callers in the `runAggregationAndSubmit` goroutine).
In Go, an unrecovered panic in a goroutine terminates the entire process. This is on the hottest path: calling for every oracle price tick from every exchange, so a single bad price from any of the 15+ exchange providers can kill the feeder process.

Affected Code
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L33

Impacts
A single bad oracle value (e.g. ticker price 0, price beyond float64 range, malformed decimal, or "NaN"/"Inf" from an exchange) triggers an unrecovered panic. The entire feeder process exits, causing full denial of service for external liquidity updates.

Remediation
Replace `MustFloat64()` with the error-returning variant; propagate the error and return it. Also, reject NaN, Inf, or non-positive price and return a clear error.

Retest
Fixed. 80669a7, PR #8. `MustFloat64()` replaced with `Float64()`; error propagated; `isValidPrice` (reject NaN, Inf, non-positive) at entry.

Bug ID #03 [Fixed]

Out-of-Bounds Panic in `collectReadyBandedData`

Vulnerability Type
Deterministic DoS/ Panic / Operator Footgun

Severity
Medium

Description
`collectReadyBandedData` uses a hardcoded loop `for i := 0; i < 5; i++` and then unconditionally accesses `poolTradeBand.Bands[i]` and `poolTradeBand.Bands[i-1]` without checking that `i` is within `len(poolTradeBand.Bands)`. This contradicts `blockchain_submitter.go` (L67–69), which states the number of bands per pool is "dynamic size now." Other components in the system already use the dynamic band configuration (for example by iterating over band slices) instead of assuming a fixed band count, so this function is the primary outlier that hardcodes the band count. If, at runtime, `len(poolTradeBand.Bands)` is less than 5 while the code still executes iterations with `i >= len(poolTradeBand.Bands)`, accessing `poolTradeBand.Bands[i]` will cause an index-out-of-range panic in the submission goroutine. This is realistic (footgun) during operator-driven band reconfiguration.

Affected Code
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/external_liquidity_manager.go#L184-L223
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/blockchain_submitter.go#L67-L69

Impacts
If the configured number of bands for a pool is less than 5 but `collectReadyBandedData` still iterates `i` from 0 to 4 & reaches an index `i` where `i >= len(poolTradeBand.Bands)`, the process will hit an index-out-of-range panic and terminate the submission path. The exact runtime conditions depend on how band configuration and aggregated scaling factor data are wired, but the unchecked hardcoded bound makes such a panic possible whenever these diverge. This is inconsistent with the rest of the system, which already relies on the dynamic band configuration rather than a fixed constant.

Remediation
Iterate over `len(poolTradeBand.Bands)`.

Retest

Fixed. c5d8ed8, PR #9. `collectReadyBandedData` iterates over `len(poolTradeBand.Bands)` instead of `hardcoded i < 5`; safe for dynamic band count.

Bug ID #04 [Fixed]

`rollingWindowDuration` Never Initialized

Vulnerability Type
Logic Error

Severity
Medium

Description
`ScalingFactorAggregator` has a `rollingWindowDuration` field intended to implement the PRD 30-minute rolling window for scaling factor data. It is used in `loadRecentData` as a time-based filter. However, `NewScalingFactorAggregator` never accepts or sets this parameter; it defaults to Go's zero value for `time.Duration`, which is 0. The guard in `loadRecentData` is `if sfa.rollingWindowDuration > 0`.
Since it is always 0, this branch is dead code and the time-based rolling window is never applied; only the count-based `windowSize` cap runs. The PRD (section 2.5) specifies a time-bounded 30-minute window; with count-only filtering, 180 samples can span less or more than 30 minutes depending on update rate, making scaling factors more volatile or stale than intended.

Affected Code
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/scaling_factor_aggregator.go#L17-L36
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/scaling_factor_aggregator.go#L191-L204

Impacts
The PRD's 30-minute rolling window is never applied, so behavior deviates from spec. With count-only filtering, scaling factors can be too volatile during bursts or too stale when updates slow down, compared to the intended design.

Remediation
Add `rollingWindowDuration` (e.g. `rollingWindow time.Duration`) to `NewScalingFactorAggregator` and set it (e.g. `30 * time.Minute`).

Retest
Fixed. 6813bb3, PR #10. `NewScalingFactorAggregator` accepts `rollingWindowDuration`; `loadRecentData` applies a time-based filter when > 0 (e.g. 30m).

Bug ID #05 [Fixed]

`stopChan` Double-Close - Manager Cannot Restart

Vulnerability Type
Lifecycle Bug / Latent Panic

Severity
Medium

Description
`BandedExternalLiquidityManager.stopChan` is a `chan struct{}` created once in the constructor and closed in `Stop()`. In Go, closing an already-closed channel causes a panic. If `Stop()` is called, then `Start()` is called again, then `Stop()` is called a second time, the second `close(elm.stopChan)` panics. Additionally, after the first `Stop()`, the channel remains closed. A subsequent `Start()` launches new goroutines that select on `elm.stopChan`; because the channel is already closed, both goroutines exit immediately on their first iteration. The manager sets `running = true` and appears operational but does no work; a silent restart failure.

Affected Code

- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/external_liquidity_manager.go#L50-L80

Impacts
A second `Stop()` after a restart causes a panic because the channel is closed twice. After the first `Stop()`, a later `Start()` makes the manager appear running but the goroutines exit immediately on the already-closed channel, so the manager does no work.

Remediation
Re-initialize `stopChan` in `Start()` before launching goroutines: e.g. `elm.stopChan = make(chan struct{})` when transitioning to running.

Retest
Fixed. 8bd0b17, PR #11. `stopChan` re-initialized in `Start()`; close only in `Stop()`; restart gets a fresh channel.

Bug ID #06 [Fixed]

Data Race on `elm.bandConfig`

Vulnerability Type
Concurrency / Data Race

Severity
Low

Description
`BandedExternalLiquidityManager` protects `elm.bandConfig` with `elm.mutex` in the writer (`UpdateBandConfig`) but reads `elm.bandConfig` in multiple functions without holding the mutex. These functions run in concurrent goroutines
(e.g. `runAggregationAndSubmit` calling `collectReadyBandedData`, and `ProcessExternalLiquidity` from oracle provider goroutines).
In Go, a struct assignment like `elm.bandConfig = newConfig` is not atomic; it writes the slice header (pointer, length, capacity) as separate memory operations. A concurrent reader can observe a partially written slice header, leading to wrong length/pointer, access to invalid/freed memory, and undefined behavior.

Affected Code
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/external_liquidity_manager.go#L89 (read without lock)
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/external_liquidity_manager.go#L185-L188 (without lock)
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/external_liquidity_manager.go#L225-L230 (correctly uses lock)

Impacts
Concurrent reads and writes of `bandConfig` form a data race that can lead to memory corruption, panics, or use of incorrect config in aggregation and submission. Failures are non-deterministic and can appear when config is updated while aggregation runs.

Remediation
Acquire `elm.mutex.RLock()` at the start of `collectReadyBandedData` and `ProcessExternalLiquidity`, release after the read of `elm.bandConfig` is complete; or take a local copy under the lock and use it below.

Retest

Fixed. 7804bea, PR #12. `bandConfig` read under `elm.mutex.RLock()` in `collectReadyBandedData` and `ProcessExternalLiquidity`; local copy then release lock.

Bug ID #07 [Fixed]

Missing defensive check for zero oracleMidPrice

Vulnerability Type
Defensive Programming / Robustness

Severity
Informational

Description
The CEX impact path (`simulateMarketSweep`) and the AMM impact path (`calculateAMMImpact`) both divide by `oracleMidPrice` without first enforcing that it is strictly positive. `oracleMidPrice` is obtained from `tPrice.Price.MustFloat64()` and can be zero in error scenarios (for example, when `GetTickerPrice` fails and returns a `TickerPrice` with `Price = 0`).
In `simulateMarketSweep`, the price impact formula `((vwap - oracleMidPrice) / oracleMidPrice) * 10000` will divide by zero if `oracleMidPrice == 0`, yielding `Inf`/`NaN`. In `calculateAMMImpact`, `baseAmountToSell := notional / oracleMidPrice` and `priceImpact = ((executionPrice / oracleMidPrice) - 1) * 10000` share the same risk. While production prices should be positive, the logic does not defensively guard against zero.

Affected Code
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L33-L36
- https://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L105-L147
- http://github.com/ryze-protocol/external-liq-feeder/blob/a355ed87fe56df611c0e9e480fb50e6d0d0b6b8c/usecase/extliq/calculate_banded_external_liquidity.go#L157-L188

Impacts
If a zero price ever leaks through, both CEX and AMM impact calculations will divide by zero, producing `Inf`/`NaN` impacts. These degenerate values can propagate into scaling factor aggregation and on-chain submissions, leading to extreme or undefined behavior. Currently, `types.TickerPrice` accepts any decimal string (including `"0"`) via `NewTickerPrice`, and the `priceStore` fallback in `GetTickerPrice` explicitly constructs a `TickerPrice` with `Price = 0` on failure; there is no global invariant enforcing strictly positive prices before they reach `CalculateBandedExternalLiquidityUseCase`. Whether upstream exchange APIs ever

return a literal zero price in practice is outside this repository's guarantees, so treat this finding as a robustness/defensive check.

Remediation

At the entry point (`CalculateBandedExternalLiquidityUseCase`), enforce `oracleMidPrice > 0` and return an error otherwise, ensuring that `simulateMarketSweep` and `calculateAMMImpact` never see zero.

Retest

Fixed. 80669a7, PR #8. `isValidPrice(oracleMidPrice)` rejects zero/non-positive at entry; no division by zero in `simulateMarketSweep` or `calculateAMMImpact`.

5.3 Signed Price Server

Bug ID #01 [Fixed]

Unsubscribe semantics inverted (empty subscription == send all)

Vulnerability Type
Logic Error

Severity
High

Description
The WebSocket message handler treats `"unsubscribe"` as "clear all subscriptions" by storing an empty symbol map, but `Client.ShouldSend` treats an empty symbol map as "send all". As a result, "unsubscribe from all" does not stop delivery and can unexpectedly increase outbound bandwidth/processing for clients that intended to stop receiving messages.The WebSocket message handler treats `"unsubscribe"` as "clear all subscriptions" by storing an empty symbol map, but `Client.ShouldSend` treats an empty symbol map as "send all". As a result, "unsubscribe from all" does not stop delivery and can unexpectedly increase outbound bandwidth/processing for clients that intended to stop receiving messages.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/types.go#L20-L35
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/hub.go#L270-L285

Impacts
Clients cannot reliably stop updates via `"unsubscribe"`. This can cause unexpected sustained traffic and server work, and can break client-side rate limiting/UX assumptions.

Remediation
Define explicit semantics: e.g., use `nil` subscription map for "send all", and an empty map (or a separate flag) for "send none". Update the `UpdateSubscription`, `"unsubscribe"` handling, and `ShouldSend` accordingly.

Retest

Fixed. 6086ec6 , PR #3. `Unsubscribe` semantics: explicit send-all sentinel vs empty map `"send none"`; `UpdateSubscription` and `ShouldSend` aligned; unsubscribe-all stops delivery.

Bug ID #02 [Fixed]

`WritePump` tight loop when `SendCh` closed (no comma-ok)

Vulnerability Type
Concurrency bug

Severity
Low

Description
On single-client disconnect, only `client.SendCh` is closed (in hub `run()` when processing Unregister); `client.CloseCh` is not closed in that path. WritePump receives from `client.SendCh` without comma-ok. When the channel is closed, receive yields zero value repeatedly and the loop never exits, causing a tight loop and CPU spike until e.g. `SendMessage` fails.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/hub.go#L285-L318
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/hub.go#L142-L151

Impacts
One goroutine per disconnecting client can spin at high CPU; DoS under frequent client churn.

Remediation
Use comma-ok: `jsonBytes, ok := <-client.SendCh; if !ok { return }`. Optionally call `client.Close()` when processing unregister so WritePump can also exit via `CloseCh`.

Retest
Fixed. 7aadc6b, PR #4. On Unregister, `client.Close()` called so WritePump exits via `CloseCh`; `WritePump` uses comma-ok on `SendCh`, returns when closed.

Bug ID #03 [Fixed]

`server.max_connections` exists but is not enforced

Vulnerability Type
Config/ Operational

Severity
Low

Description
`Server.MaxConnections` is defined in config (`max_connections`, default 10000 in SetDefaults and TOML) but is never passed to the hub or API server. `NewHub` accepts only `ctx`, `broadcastQueueSize`, and `logger`; in `run()` the hub accepts every client on `h.register` with no check against a limit. `NewServer` and `handleWebSocket` do not receive or use `MaxConnections`. As a result the setting has no effect and concurrent WebSocket connections are unbounded; operators may believe they are capping connections when they are not.

Affected Code

- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go#L49
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/hub.go#L58-L69 (`NewHub` does not take max connections; register case accepts without cap check)
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/api/handlers.go#L23-L42 (server and WebSocket handler do not use `MaxConnections`)

Impacts
Config is misleading; operators may expect a connection cap that is not applied. In current deployment (single trusted relayer) not exploitable; recommendation for clarity or future hardening.

Remediation
Pass `MaxConnections` into the hub and reject or back-pressure when `connCount >= MaxConnections` in the register path; or remove the option from config and document that connections are not capped.

Retest

Fixed. 5d39beb, PR #5. `MaxConnections` passed into hub; register path rejects or back-pressures when `conn count >= MaxConnections`.

Bug ID #04 [Fixed]

Config defaults override explicit `false` (cannot disable)

Vulnerability Type
Config Error

Severity
Low

Description
`SetDefaults()` in `config/types.go` only fills zero values for strings/ints, preserving explicit user settings. For `Logging.Timestamp` and `Metrics.Enabled` it does `if !x { x = true }`. A plain `bool` in Go has zero value `false`, so "key omitted" and "key set to false" are indistinguishable after unmarshal; SetDefaults overwrites both with `true`. The config file presents these as toggles (`# Enable timestamp in logs`, `# Enable Prometheus metrics`), so the intended contract is user-configurable; the implementation cannot preserve explicit `false`.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go#L210-L220
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/config.toml#L168-L174

Impacts
Operators who set `logging.timestamp = false` or `metrics.enabled = false` in config (e.g. for compliance, log pipelines, or reducing exposure) find the process still behaves as if they were `true`. This is an operational/behavioral mismatch and can cause confusion or compliance/logging issues.

Remediation
Use pointers (`*bool`) to distinguish "unset" from "explicit false" and only set the default when the pointer is nil; or detect whether the config key was present and only apply defaults when absent.

Retest
Fixed. a41864c, PR #6. Config cleanup; some bool/default handling improved.

Bug ID #05 [Acknowledged]

BinanceUS endpoint overrides are ignored

Vulnerability Type
Config Error

Severity
Low

Description
`NewBinanceProvider` overwrites the passed `endpoints` whenever `endpoints.Name != ProviderBinance`. For BinanceUS (`binanceUS=true`), even correctly named endpoints (`ProviderBinanceUS`) will still be `!= ProviderBinance`, so config-provided Rest/Websocket values are ignored.

Affected Code

- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/provider/binance.go#L129-L142

Impacts
Operators cannot route `BinanceUS` through custom endpoints (proxies, region-specific endpoints, failover), reducing reliability and configurability.

Remediation
Only apply defaults when Rest/Websocket fields are empty, and validate against the expected provider name for the `binanceUS` mode.

Retest
Acknowledged. No patch needed. `NewBinanceProvider` still overwrites endpoints when `endpoints.Name != ProviderBinance`; BinanceUS custom Rest/Websocket from config ignored.

Bug ID #06 [Fixed]

Precision loss and truncation in priceToWad (IEEE-754)

Vulnerability Type
Incorrect calculation / decimal normalization

Severity
Low

Description
`priceToWad` uses `strconv.ParseFloat(priceStr, 64)`, limiting precision to ~15–17 decimal digits, and `big.Float.Int(priceInWad)` truncates toward zero (no rounding). Signed `priceInWad` is used in `keccak256(abi.encodePacked(token, priceInWad, timestamp))` and on-chain math. However, Binance prices have at most 8 decimal places (e.g. `"67432.12000000"`), well within float64's ~15-digit precision, so the intermediate conversion introduces at most sub-wei rounding error. Impact is further bounded by contract pDelta (e.g. ≤0.5% deviation from Pyth oracle). The truncation bias is ≤1 wei. Fix is recommended for correctness best practice, but actual on-chain impact is negligible.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/cmd/bot/main.go#L170-L189

Impacts
Theoretically incorrect signed prices used on-chain, but in practice Binance's ≤8 decimal-place prices are well within float64 precision. At worst, sub-wei rounding error in the 18th decimal place; orders of magnitude below pDelta tolerance. No realistic economic loss.

Remediation
Parse price as `big.Float` from string (e.g. `SetString`) to avoid float64; use explicit rounding for WAD conversion and document rounding rule; reject or cap negative/zero/out-of-range prices .

Retest
Fixed. 83a9dde, PR #7. `priceToWad: big.Float/SetString` for precision; explicit rounding for WAD; invalid/zero/out-of-range prices rejected or capped.

Bug ID #07 [Fixed]

consumeSignedPrices infinite loop when SignedPrices channel closed (no comma-ok)

Vulnerability Type
Concurrency bug

Severity
Low

Description
`workerPool.Stop()` closes `wp.signedQueue` after workers exit. The `consumeSignedPrices` goroutine uses main's `ctx`, which is cancelled only when main returns (defer cancel()), i.e. after `workerPool.Stop()` has already closed the channel. The loop receives from `workerPool.SignedPrices()` without comma-ok, so when the channel is closed it repeatedly receives zero value and never exits, causing a busy loop and possible stray zero-value writes to store/hub until process exit.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/cmd/bot/main.go#L191-L231
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/signing/worker_pool.go#L101-L111

Impacts
Shutdown correctness; brief busy loop between channel close and context cancellation (sub-millisecond window in practice, since `defer cancel()` fires immediately after `main` returns). Zero-value reads during this window could store/broadcast an empty `SignedPrice`, but the window is too short for meaningful impact.

Remediation
Use comma-ok: `price, ok := <-workerPool.SignedPrices(); if !ok { return }`. Optionally cancel main's ctx before calling `workerPool.Stop()` so the consumer exits on `ctx.Done()` before the channel is closed.

Retest
Fixed. ccbf830, PR #8. `consumeSignedPrices` uses comma-ok on `SignedPrices()`; if !ok (channel closed) returns; no busy loop.

Bug ID #08 [Fixed]

shutdown_timeout numeric/format mismatch (config silently ignored)

Vulnerability Type
Config Semantics

Severity
Low

Description
Config may use an integer for shutdown timeout (e.g. `shutdown_timeout = 30` in TOML). The struct expects a `ShutdownTimeout string`. Viper may unmarshal the int as the string `"30"`. `time.ParseDuration("30")` fails because it requires a unit suffix (e.g. `"30s"`, `"1m"`). The code then falls back to a hardcoded `30 * time.Second`, so the user's numeric value is silently ignored unless they use a duration string (e.g. `"30s"`).

Affected Code

- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/config.toml#L14-L15 (`shutdown_timeout= 30` (integer))
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go#L52 (`ShutdownTimeout` string)
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go#L173-L174 (SetDefaults: `"30s"` when empty)
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/cmd/bot/main.go#L156-L160 (`ParseDuration(cfg.Server.ShutdownTimeout)`; on error fallback to 30s)

Impacts
Config value silently ignored when not a valid duration string; misleading configuration.

Remediation
Document that `shutdown_timeout` must be a duration string (e.g. `"30s"`); or accept numeric seconds and append `"s"` before parsing; validate at load and return error for invalid values.

Retest
Fixed. 2f38885, PR #9. `shutdown_timeout` parsing fixed; normalize/validate duration; invalid values handled; format documented.

Bug ID #09 [Fixed]

`token_mappings` address not hex-validated

Vulnerability Type
Missing Validation/ Config Footgun

Severity
Low

Description
`TokenMappingsMap` uses `common.HexToAddress(mapping.Address)`; `HexToAddress` returns the zero address for invalid hex and does not error. A typo or malformed address in config becomes `0x0` and can produce wrong or dangerous signatures. Contract still verifies the signer; impact is wrong token mapping, not theft.

Affected Code

- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go#L254-L261
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/config.toml

Impacts
Invalid or malformed addresses in `token_mappings` config become zero or wrong token with no error at config load. The trade callback uses this map to sign Binance trades; wrong token is embedded in the signature and sent by the relayer to the contract. So, the contract may revert or apply price to the wrong asset.

Remediation
Validate each `mapping.Address` with `common.IsHexAddress` (or equivalent) and return an error from config load if invalid; reject zero address unless explicitly allowed.

Retest
Fixed. 0a67330, PR #10. `token_mappings` address validated with `common.IsHexAddress` at config load; invalid/zero rejected with error.

Bug ID #10 [Fixed]

WorkerPool.Stop() not idempotent

Vulnerability Type
Logic Error

Severity
Informational

Description
`WorkerPool.Stop()` closes `wp.signedQueue` with no guard. If `Stop()` is called twice (e.g. tests, shutdown paths, or refactors), the second `close(wp.signedQueue)` panics with "close of closed channel". The main process currently calls `Stop()` only once, but the contract is fragile.

Affected Code

- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/signing/worker_pool.go#L100-L111

Impacts
Panic during shutdown if `Stop()` is invoked more than once; harder to reuse or test the worker pool safely.

Remediation
Make `Stop()` idempotent (e.g. `sync.Once` or a "stopped" flag) and only close `signedQueue` once; document that `Stop()` is safe to call multiple times.

Retest
Fixed. e9c987a, PR #11. `WorkerPool.Stop()` uses `sync.Once`; signedQueue closed once; idempotent and safe to call multiple times.

Bug ID #11 [Fixed]

Multiple configuration variables are defined but unused (dead config)

Vulnerability Type
Dead Code

Severity
Informational

Description
Several configuration variables are parsed and sometimes set with defaults, but never actually used in the codebase:

1. `Server.ReadTimeout` and `Server.WriteTimeout`: The HTTP server hardcodes `ReadTimeout: 10 * time.Second`, `WriteTimeout: 0`.
2. `Hub.PingInterval` and `Hub.WriteTimeout`: The hub uses hardcoded timers (e.g. `30s` ticker, `10s` write deadline).
3. `Server.ReadBufferSize` and `Server.WriteBufferSize`: The WebSocket upgrader hardcodes `1024` for both.
4. `ProviderTimeout`: Parsed but never used in any provider or websocket code.

Operators may believe they are tuning performance or timeouts, leading to a false sense of security or misconfiguration risk when these limits are not actually enforced.

Affected Code
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/config/types.go (`ReadTimeout`, `WriteTimeout`, `PingInterval`, `ReadBufferSize`, `WriteBufferSize`, `ProviderTimeout` parsing and SetDefault)
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/api/server.go#L46-L54
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/stream/hub.go#L291
- https://github.com/ryze-protocol/signed-price-server/blob/719efe5adca5957fc848ee778eb956860e461c52/api/handlers.go#L14-L19

Impacts
Dead configuration surface causing confusion. Future re-enabling or assumptions about limits might introduce bugs or operational issues.

Remediation

Prefer implementing the configuration values if runtime tuning of timeouts and buffers is a design goal. Otherwise, remove the unused fields entirely to reduce configuration surface area and prevent a false sense of security.

Retest

Fixed. a41864c, PR #6. Config cleanup removed some unused fields.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as blockchain infrastructure, backend signing services, and on-chain execution systems carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com